SPA Quiz App Guide

React + TypeScriptで学ぶSPA開発とクイズUI実装

2026-03-21

React SPA 選択問題アプリ

学習用クイズアプリ実践ガイド

TypeScript + React + Vite + Tailwind CSS を用いた学習用クイズ SPA の実装・運用・教材作成フローをまとめたガイドです。

- 著者: 杜 文吉
- 対象: フロントエンド学習者 / React 開発者 / 教材作成者
- 公開サイト: https://duwenji.github.io/spa-quiz-app/

収録内容

- クイズ受験者向けの利用ガイド
- クイズ作成者向けの取り込み・運用ガイド
- クイズデータ仕様と命名ルール

目次

- 第1章 利用ガイド (#chapter-01-user-guide)
  - 1.1 クイズ受験マニュアル (#section-01-user-guide-01-quiz-taker)
- 第2章 クイズ作成と運用 (#chapter-02-content-authoring)
  - 2.1 クイズサイト取り込みマニュアル (#section-02-content-authoring-01-quiz-creator)
- 第3章 技術仕様 (#chapter-03-technical-specification)
  - 3.1 クイズデータ仕様書 (#section-03-technical-specification-01-quiz-spec)

第1章 利用ガイド

1.1 クイズ受験マニュアル

> **対象:** ブラウザでクイズサイトにアクセスして学習する方

1.1.1 目次

1. サイトへのアクセス (#1-サイトへのアクセス)
2. クイズの選び方 (#2-クイズの選び方)
3. クイズを始める (#3-クイズを始める)
4. 問題への回答 (#4-問題への回答)
5. ナビゲーション機能 (#5-ナビゲーション機能)
6. 学習統計とダウンロード (#6-学習統計とダウンロード)
7. よくある質問 (#7-よくある質問)

1.1.2 サイトへのアクセス

ブラウザで以下の URL を開いてください。

```
https://duwenji.github.io/spa-quiz-app/
```

アカウント登録や会員登録は不要です。ブラウザのみで利用できます。

1.1.3 クイズの選び方

サイトを開くと、利用可能なクイズセットの一覧が表示されます。

1.1.3.1 一覧の見方

各カードには以下の情報が表示されます。

項目	説明
アイコン	クイズのカテゴリを表す絵文字
タイトル	クイズセットの名前
説明	学習できる内容の概要
問題数	収録されている問題の数
難易度	beginner / intermediate / advanced など

1.1.3.2 シリーズ（ステップ）形式のクイズ

「クリーンアーキテクチャ完全ガイド」や「GitHub Copilot Agent & Skills 総合シリーズ」「GitHub Copilot 実務ワークフローシリーズ」など、複数のステップに分かれているクイズは、一覧画面で**シリーズ見出し付きの個別ステップ**として表示されます。

受験の際は表示されている個別のステップをそのまま選択してください。

1.1.4 クイズを始める

クイズカードをクリックすると、イントロ画面が表示されます。

1.1.4.1 イントロ画面で確認できること

- クイズの正式名称
- 学習内容の説明
- 問題数を含むクイズ概要

「**クイズを始める**」ボタンをクリックするとクイズが開始されます。

「 **<- 戻る**」をクリックするとクイズ一覧に戻ります。

1.1.5 問題への回答

1.1.5.1 回答手順

1. 問題文を読む

2. A / B / C / D の選択肢のいずれかをクリックして選択する
3. 「**確認する**」ボタンをクリックする

> **注意:** 「確認する」をクリックしないと回答が記録されません。

1.1.5.2 結果フィードバック

回答を確定すると、その場で正誤が表示されます。

- **正解の場合:** 緑色の表示と解説が表示されます
- **不正解の場合:** 赤色の表示・正解・解説が表示されます

解説を読んで理解を深めてから、次の問題に進んでください。

1.1.5.3 プログレスバー

画面上部のプログレスバーで、現在のクイズ全体の進捗（正解数・不正解数）を確認できます。

1.1.6 ナビゲーション機能

1.1.6.1 前後移動

- 「**次の問題**」: 次の問題に進む（回答確定後に使用可）
- 「**前の問題**」: 前の問題に戻る

前の問題に戻った場合、以前の回答が自動的に復元されます。
回答済みの問題は変更できません。

1.1.6.2 問題番号ジャンプ

画面下部の問題番号リストをクリックすると、任意の問題に直接移動できます。

マーク	意味
[OK] 緑	正解した問題
赤	不正解だった問題
グレー	未回答の問題
青枠	現在表示中の問題

1.1.7 学習統計とダウンロード

クイズ中に1問以上回答すると、画面下部に「 学習統計とダウンロード」パネルが表示されます。

1.1.7.1 表示される統計

統計	説明
正解数	正解した問題の数 / 回答した問題数
正解率	全回答に対する正解の割合（%）
平均解答時間	1問あたりの平均所要時間（秒）
回答数	回答済みの問題数

初回回答日時・最終回答日時も確認できます。

1.1.7.2 データのダウンロード

「 **JSON形式でダウンロード**」
すべての回答記録をJSON形式のファイルとして保存します。プログラムでの分析に適しています。

「 **CSV形式でダウンロード**」
回答記録をCSV形式で保存します。Excelや Google スプレッドシートで開けます。

1.1.7.3 AI分析（Copilot Chat）

「**[Checklist] AI分析用プロンプトをコピー**」ボタンをクリックすると、学習データの分析を依頼するプロンプトがクリップボードにコピーされます。

VS Code の Copilot Chat（`Ctrl+Shift+I`）を開いてプロンプトを貼り付けると、苦手分野の特定や学習アドバイスを得られます。

1.1.7.4 学習履歴のリセット

「**[Update] 学習履歴をリセット**」ボタンをクリックすると、現在のセッションの回答記録がすべて削除されます。

> **注意:** リセットすると履歴を元に戻せません。必要に応じてダウンロードしてから実行してください。

1.1.7.5 学習履歴の保存場所

学習履歴はブラウザの **ローカルストレージ** に自動保存されます。
同じブラウザ・同じデバイスで**同じクイズセット**を再度開くと、回答済み履歴が自動復元され、未回答の続きから再開できます。別のブラウザや別のデバイスには引き継がれません。

1.1.8 よくある質問

1.1.8.1 Q. 問題が表示されない / 「データが見つかりません」と表示される

「クイズ一覧に戻る」ボタンをクリックして別のクイズセットを選んでください。
問題が解消されない場合は、ブラウザのキャッシュをクリアしてください
（`Ctrl+Shift+Delete`）。

1.1.8.2 Q. 途中でブラウザを閉じると進捗は消えますか？

回答済みの記録はローカルストレージに保存されます。 同じブラウザ・同じデバイスで同じクイズセットを開き直すと、続きから再開できます。別のブラウザや別のデバイスへの引き継ぎはできません。
長期保存したい場合は JSON または CSV でダウンロードしてください。

1.1.8.3 Q. 回答を後から修正できますか？

回答を確定した問題は変更できません。問題番号からその問題を表示することはできますが、選択肢を変更することはできない仕様です。

1.1.8.4 Q. ページ表示が古いままに見える

`Ctrl+Shift+R`（強制リロード）を試してください。
それでも改善しない場合はブラウザキャッシュのクリアをお試しください
（`Ctrl+Shift+Delete`）。

1.1.8.5 Q. スマートフォンでも使えますか？

はい、レスポンシブデザインのため PC・タブレット・スマートフォンで利用できます。

第2章 クイズ作成と運用

2.1 クイズサイト取り込みマニュアル

> **対象:** クイズコンテンツを新規作成・追加してサイトに反映させる方

> [Checklist] データ形式の詳細仕様は **クイズデータ仕様書** (#section-03-technical-specification-01-quiz-spec) を参照してください（開発環境なしで外部からコンテンツを提供する方向け）。

2.1.1 目次

1. はじめに (#1-はじめに)
2. 環境セットアップ (#2-環境セットアップ)
3. クイズデータの形式 (#3-クイズデータの形式)
4. クイズセットの作成手順 (#4-クイズセットの作成手順)
5. AI を使った問題生成 (#5-ai-を使った問題生成)
6. バリデーション (#6-バリデーション)
7. デプロイ (#7-デプロイ)
8. アーキテクチャ参考 (#8-アーキテクチャ参考)
9. トラブルシューティング (#9-トラブルシューティング)

2.1.2 はじめに

2.1.2.1 取り込みの全体フロー

```
1.  src/data/ に問題 JSON ファイルを作成
        down
2.  src/data/quizSets.json にメタデータを登録
        down
3.  必要に応じて sync-data / build を実行
        down
4.  バリデーション実行
        down
5.  npm run build && npm run deploy
   down
6.  GitHub Pages に反映
```

2.1.2.2 ファイルの配置ルール

用途	場所
問題 JSON のソース	`src/data/<カテゴリ>/<quiz-id>.json`
メタデータ一覧のソース	`src/data/quizSets.json`
実行時に参照される公開データ	`public/data/`
JSON スキーマ	`public/schemas/`

> このリポジトリでは **`src/data/` が編集元** です。`npm run build` 実行時に `npm run sync-data` が走り、`src/data/` の内容が `public/data/` にコピーされます。

2.1.3 環境セットアップ

2.1.3.1 必要なもの

- Node.js v16 以上
- Git

- GitHub アカウント（デプロイする場合）

2.1.3.2 初回セットアップ

```
(#cb3-1)# リポジトリをクローン
(#cb3-2)git clone https://github.com/duwenji/spa-quiz-app.git
(#cb3-3)cd spa-quiz-app
(#cb3-4)
(#cb3-5)# パッケージをインストール
(#cb3-6)npm install
(#cb3-7)
(#cb3-8)# 開発サーバーを起動（http://localhost:5173）
(#cb3-9)npm run dev
```

2.1.3.3 主要コマンド

コマンド	説明	環境
`npm run dev`	開発サーバー起動（HMR 有効）	localhost:5173
`npm run sync-data`	`src/data/` を `public/data/` にコピー	ローカル確認前
`npm run build`	本番ビルド -> `dist/` を生成	ローカル確認
`npm run preview`	ビルド済み `dist/` を確認	localhost:4173
`npm run deploy`	ビルド + GitHub Pages へ公開	本番
`npm run validate:metadata`	`quizSets.json` を検証	開発
`npm run validate:quiz`	設定済み glob に一致する問題 JSON を検証	開発

2.1.4 クイズデータの形式

2.1.4.1 1. 問題 JSON ファイル（`<quiz-id>.json`）

```
(#cb4-1)  {
(#cb4-2)    "questions": [
(#cb4-3)      {
(#cb4-4)        "id": 1,
(#cb4-5)        "question": "問題文をここに記述する",
(#cb4-6)        "options": [
(#cb4-7)          { "id": "A", "text": "選択肢A" },
(#cb4-8)          { "id": "B", "text": "選択肢B" },
(#cb4-9)          { "id": "C", "text": "選択肢C" },
(#cb4-10)          { "id": "D", "text": "選択肢D" }
(#cb4-11)        ],
(#cb4-12)        "correctAnswer": "A",
(#cb4-13)        "explanation": "解説文。なぜAが正解なのかを詳しく説明する。"
(#cb4-14)      },
(#cb4-15)      {
(#cb4-16)        "id": 2,
(#cb4-17)        "question": "問題文をここに記述する",
(#cb4-18)        "options": [
(#cb4-19)          { "id": "A", "text": "選択肢A" },
(#cb4-20)          { "id": "B", "text": "選択肢B" },
(#cb4-21)          { "id": "C", "text": "選択肢C" },
(#cb4-22)          { "id": "D", "text": "選択肢D" }
(#cb4-23)        ],
(#cb4-24)        "correctAnswer": "B",
(#cb4-25)        "explanation": "解説文。"
(#cb4-26)      }
(#cb4-27)    ]
(#cb4-28)  }
```

2.1.4.1.1 フィールド仕様

フィールド	型	必須	説明
`id`	number	[OK]	クイズ内で 1 から始まる連番
`question`	string	[OK]	問題文
`options`	array	[OK]	選択肢（A / B / C / D の 4 つ必須）
`options[].id`	string	[OK]	`"A"` / `"B"` / `"C"` / `"D"` のいずれか
`options[].text`	string	[OK]	選択肢の本文
`correctAnswer`	string	[OK]	正解（`"A"` / `"B"` / `"C"` / `"D"`）
`explanation`	string	[OK]	解説（空文字禁止）

2.1.4.1.2 問題作成チェックリスト

- ❒ `id` が 1 から始まる連番になっている
- ❒ `options` が A / B / C / D の 4 つすべてある
- ❒ `correctAnswer` が `options` のいずれかと一致している
- ❒ `explanation` が空でない
- ❒ すべての文字列フィールドが空でない

2.1.4.1.3 難易度の目安

- **beginner（15%）**：用語の定義、基本コマンドの目的
- **intermediate（70%）**：設定内容の理解、エラー解決

- **advanced（15%）**：システム全体の把握、複合的な問題分析

2.1.4.2 2. メタデータ（`quizSets.json`）

`src/data/quizSets.json` に、問題 JSON ファイルの情報を登録します。

```
(#cb5-1)[
(#cb5-2)  {
(#cb5-3)    "id": "my-new-quiz",
(#cb5-4)    "name": "新しいクイズセット",
(#cb5-5)    "description": "このクイズで学べる内容の概要を50～200文字で記述します。",
(#cb5-6)    "category": "技術",
(#cb5-7)    "icon": "[Note] ",
(#cb5-8)    "questionCount": 20,
(#cb5-9)    "difficulty": "intermediate",
(#cb5-10)    "dataPath": "my-category/my-new-quiz.json",
(#cb5-11)    "parentId": null,
(#cb5-12)    "group": null,
(#cb5-13)    "level": 1,
(#cb5-14)    "order": 1
(#cb5-15)  }
(#cb5-16)]
```

2.1.4.2.1 メタデータフィールド仕様

フィールド	型	必須	説明
`id`	string	[OK]	一意の ID（kebab-case）例: `"github-copilot-variables"`
`name`	string	[OK]	クイズ一覧に表示される名前
`description`	string	[OK]	学習内容の概要（50〜200 文字）
`category`	string	[OK]	カテゴリ名（例: `"GitHub"`, `"アーキテクチャ"`, `"技術"`）
`icon`	string	[OK]	絵文字アイコン（例: `" "`, `" "`, `"[Guide] "`）
`questionCount`	number	[OK]	問題数（問題 JSON の実際の件数と一致させる）
`difficulty`	string	[OK]	`"beginner"` / `"intermediate"` / `"advanced"` / `"beginner to intermediate"` / `"beginner to advanced"`
`dataPath`	string	[OK]	実行時の `public/data/` からの相対パス。ソースは `src/data/` 側で管理する（例: `"github-copilot/variables.json"`）
`parentId`	string|null	[OK]	親クイズの ID（階層構造を持つ場合）/ 単体なら `null`

フィールド	型	必須	説明
`group`	string|null	[OK]	シリーズのグループ識別子 / 単体なら `null`
`level`	number	[OK]	`1` = 単体またはシリーズ親, `2` = シリーズの子
`order`	number	[OK]	グループ内の表示順

2.1.4.2.2 命名規則

対象	規則	例
クイズ ID	kebab-case（小文字・ハイフン）	`github-copilot-variables`
JSON ファイル名	kebab-case.json	`github-copilot-variables.json`
フォルダ名	小文字・ハイフン	`github-copilot/`

2.1.4.3 3. 階層構造（シリーズ形式）

複数ステップからなるシリーズを作る場合：

```
(#cb6-1)[
(#cb6-2)  {
(#cb6-3)    "id": "my-series",
(#cb6-4)    "name": "My Series（全体）",
(#cb6-5)    "description": "シリーズ全体の概要",
(#cb6-6)    "category": "技術",
(#cb6-7)    "icon": "[Guide] ",
(#cb6-8)    "questionCount": 0,
(#cb6-9)    "difficulty": "intermediate",
(#cb6-10)    "dataPath": "",
(#cb6-11)    "parentId": null,
(#cb6-12)    "group": "my-series",
(#cb6-13)    "level": 1,
(#cb6-14)    "order": 1
(#cb6-15)  },
(#cb6-16)  {
(#cb6-17)    "id": "my-series-step1",
(#cb6-18)    "name": "Step 1: 入門",
(#cb6-19)    "description": "Step 1 の概要",
(#cb6-20)    "category": "技術",
(#cb6-21)    "icon": "[Guide] ",
(#cb6-22)    "questionCount": 15,
(#cb6-23)    "difficulty": "beginner",
(#cb6-24)    "dataPath": "my-category/my-series-step1.json",
(#cb6-25)    "parentId": "my-series",
(#cb6-26)    "group": "my-series",
(#cb6-27)    "level": 2,
(#cb6-28)    "order": 1
(#cb6-29)  },
(#cb6-30)  {
(#cb6-31)    "id": "my-series-step2",
(#cb6-32)    "name": "Step 2: 応用",
(#cb6-33)    "description": "Step 2 の概要",
(#cb6-34)    "category": "技術",
(#cb6-35)    "icon": "[Guide] ",
(#cb6-36)    "questionCount": 20,
(#cb6-37)    "difficulty": "intermediate",
(#cb6-38)    "dataPath": "my-category/my-series-step2.json",
(#cb6-39)    "parentId": "my-series",
(#cb6-40)    "group": "my-series",
(#cb6-41)    "level": 2,
(#cb6-42)    "order": 2
```

```
(#cb6-43)  }
(#cb6-44)]
```

2.1.5 クイズセットの作成手順

2.1.5.1 Step 1: 問題 JSON ファイルを作成する

1. `src/data/<カテゴリフォルダ>/` を作成する（フォルダがなければ）
2. `<quiz-id>.json` を作成し、フォーマット (#3-1-問題-json-ファイルquiz-idjson) に従って問題を記述する

```
src/
  data/
    my-category/            <-  新規フォルダ（必要に応じて）
      my-new-quiz.json      <-  新規ファイル
```

2.1.5.2 Step 2: quizSets.json にメタデータを追加する

`src/data/quizSets.json` を編集し、既存の配列末尾に新しいエントリを追加します。

```
(#cb8-1){
(#cb8-2)  "id": "my-new-quiz",
(#cb8-3)  "name": "クイズのタイトル",
(#cb8-4)  "description": "学習内容の概要（50～200文字）",
(#cb8-5)  "category": "技術",
(#cb8-6)  "icon": "[Note] ",
(#cb8-7)  "questionCount": 20,
(#cb8-8)  "difficulty": "intermediate",
(#cb8-9)  "dataPath": "my-category/my-new-quiz.json",
(#cb8-10)  "parentId": null,
(#cb8-11)  "group": null,
(#cb8-12)  "level": 1,
(#cb8-13)  "order": 1
(#cb8-14)}
```

2.1.5.3 Step 3: バリデーションを実行する

```
(#cb9-1)# メタデータの検証
(#cb9-2)npm run validate:metadata
(#cb9-3)
(#cb9-4)# 公開データを更新
(#cb9-5)npm run sync-data
(#cb9-6)
(#cb9-7)# 問題 JSON の検証（単一ファイル指定）
(#cb9-8)npm run validate:quiz -- src/data/my-category/my-new-quiz.json
(#cb9-9)
(#cb9-10)# 設定ファイルの questionGlobs に一致するファイルをまとめて検証
(#cb9-11)npm run validate:quiz
```

複数ファイルをまとめて検証対象に含めるには、`.github/skills-config/quiz-generator/quiz-generator.config.json` の `questionGlobs` を更新してください。

エラーが出た場合は バリデーション (#6-バリデーション) セクションを参照してください。

2.1.5.4 Step 4: ローカルで動作確認する

```
(#cb10-1)npm run sync-data
(#cb10-2)npm run dev
```

`http://localhost:5173` を開き、追加したクイズが一覧に表示されることを確認します。

2.1.5.5 Step 5: デプロイする

動作確認後、デプロイ (#7-デプロイ) セクションの手順に従ってデプロイします。

2.1.6 AI を使った問題生成

GitHub Copilot などの AI アシスタントを使うと、ドキュメントや教材から効率的に問題を生成できます。

2.1.6.1 プロンプト例

以下のプロンプトをコピーして Copilot Chat に貼り付けてください。

```
以下のドキュメントをもとに、クイズ問題を20問作成してください。

# 形式
- JSON形式（questions 配列）
- 各問題に id（連番）、question、4択の options（A/B/C/D）、correctAnswer、explanation を含める
- 難易度: beginner 15%、intermediate 70%、advanced 15%
- 解説は詳しく（なぜその選択肢が正答か）

# 対象ドキュメント
（ここにドキュメントの内容を貼り付ける）
```

2.1.6.2 生成した問題の品質チェックポイント

- [ ] 問題文が明確で曖昧さがない
- [ ] 不正解選択肢もそれぞれ意味のある内容になっている
- [ ] 正解が `options` 内の ID と一致している
- [ ] 解説がドキュメントの内容を正確に反映している
- [ ] `id` が 1 からの連番になっている

2.1.7 バリデーション

2.1.7.1 メタデータの検証

```
(#cb12-1)npm run validate:metadata
```

`src/data/quizSets.json` を JSON スキーマ（`public/schemas/quizset-metadata-schema.json`）で検証します。

2.1.7.2 問題 JSON の検証

```
(#cb13-1)# 特定ファイルを検証
(#cb13-2)npm run validate:quiz -- src/data/my-category/my-new-quiz.json
(#cb13-3)
(#cb13-4)# 設定ファイルの questionGlobs に一致する問題ファイルを検証
(#cb13-5)npm run validate:quiz
```

複数ファイルを一括検証したい場合は、`.github/skills-config/quiz-generator/quiz-generator.config.json` の `questionGlobs` を更新してください。

2.1.7.3 主なエラーメッセージと対処法

エラー	原因	対処
`"id" is required`	id フィールドが未設定	メタデータに id を追加
`"options" must have 4 items`	選択肢が 4 つない	A/B/C/D をすべて追加
`"correctAnswer" must be one of A, B, C, D`	correctAnswer の値が不正	大文字のA/B/C/Dに修正
`"questionCount" does not match actual count`	問題数と questionCount が不一致	quizSets.json の questionCount を修正
`"description" must be 50-200 characters`	説明文が短すぎる or 長すぎる	文字数を調整

2.1.8 デプロイ

2.1.8.1 前提条件

- GitHub リポジトリへのプッシュ権限があること
- GitHub Pages の設定で `gh-pages` ブランチが選択されていること（`Settings > Pages > Branch: gh-pages`）

2.1.8.2 デプロイ手順

```
(#cb14-1)# 1. ローカルでビルドして確認
(#cb14-2)npm run build
(#cb14-3)npm run preview   # http://localhost:4173 で本番相当の動作を確認
(#cb14-4)
(#cb14-5)# 2. GitHub Pages にデプロイ
(#cb14-6)npm run deploy
```

`Published` と表示されれば成功です。

2.1.8.3 デプロイ後の確認

URL にアクセスして追加したクイズが表示されることを確認します。

```
https://duwenji.github.io/spa-quiz-app/
```

> **反映タイミング**: デプロイ後、GitHub Pages への反映には数分かかることがあります。

2.1.8.4 vite.config.ts の base 設定

本番と開発でパスが異なる設定が自動的に適用されます。手動変更は不要です。

```
(#cb16-1)// vite.config.ts（参考）
(#cb16-2)base: process.env.NODE_ENV === 'production' ? '/spa-quiz-app/' : '/'
```

2.1.9 アーキテクチャ参考

2.1.9.1 データの流れ

```
src/data/index.ts
     getAllQuizSets()       <-  src/data/quizSets.json を読み込む
     getQuizSetQuestions()  <-  quizSet.dataPath を fetch して問題配列を返す

src/data/
  quizSets.json             <-  編集元のメタデータ一覧
  github-copilot/
    github-copilot-variables.json
  clean-architecture/
    step1-foundation.json
    ...

build / sync-data 実行後
public/data/
  ...                       <-  実行時に fetch される公開データ
```

2.1.9.2 ビルド出力と GitHub Pages

```
npm run build
     dist/
       index.html
       assets/           <-  JS / CSS バンドル
       data/             <-  public/data/ がそのままコピーされる
       schemas/          <-  public/schemas/ がそのままコピーされる
```

`dist/` の内容が `gh-pages` ブランチにプッシュされ、GitHub Pages がホストします。

2.1.10 トラブルシューティング

2.1.10.1 ビルドエラー

2.1.10.1.1 `Property 'env' does not exist on type 'ImportMeta'`

`tsconfig.json` に以下を追加してください。

```
(#cb19-1){
(#cb19-2)  "compilerOptions": {
(#cb19-3)    "types": ["vite/client"]
(#cb19-4)  }
(#cb19-5)}
```

2.1.10.1.2 `Module not found: src/data/`

`public/data/` が存在しない場合に発生します。PowerShell で以下を実行して作成してください。

```
(#cb20-1)Copy-Item -Recurse src/data/ public/data/
```

2.1.10.2 デプロイエラー

2.1.10.2.1 `npm run deploy` が失敗する

以下を確認してください。

1. `gh-pages` パッケージがインストールされているか

```
(#cb21-1)npm install gh-pages --save-dev
```

2. Git のユーザー設定がされているか

```
(#cb22-1)git config --global user.email "you@example.com"
(#cb22-2)git config --global user.name "Your Name"
```

3. GitHub 認証が完了しているか（SSH キーまたは Personal Access Token）

2.1.10.2.2 GitHub Pages に古いバージョンが表示される

- ブラウザキャッシュをクリア（`Ctrl+Shift+Delete`）してから再確認
- GitHub リポジトリの `gh-pages` ブランチのコミット日時を確認
- それでも解消しない場合は再度 `npm run deploy` を実行

2.1.10.3 データが表示されない

2.1.10.3.1「データが見つかりません」エラー

1. `public/data/` に問題 JSON ファイルが存在するか確認する
2. `quizSets.json` の `dataPath` が実際のファイルパスと一致しているか確認する
3. 開発サーバーのブラウザコンソール（F12）でネットワークエラーを確認する

2.1.10.3.2 ローカル開発中に 404 エラーが出る

`public/` フォルダと `public/data/` サブフォルダが存在することを確認してください。

```
(#cb23-1)# フォルダ構成を確認
(#cb23-2)Get-ChildItem public/data/ -Recurse
```

第3章 技術仕様

3.1 クイズデータ仕様書

> **対象:** 外部からクイズコンテンツを提供・取り込む方

本ドキュメントは、spa-quiz-app に取り込む**クイズデータの形式仕様**を定義します。
開発環境がなくても、本仕様に従ったファイルを作成すれば取り込みを依頼できます。

3.1.1 目次

1. ファイル構成 (#1-ファイル構成)
2. 問題ファイル仕様 (#2-問題ファイル仕様)
3. メタデータ仕様 (#3-メタデータ仕様)
4. シリーズ（階層）構成 (#4-シリーズ階層構成)
5. 命名規則 (#5-命名規則)
6. 難易度ガイドライン (#6-難易度ガイドライン)
7. バリデーション (#7-バリデーション)
8. 提出方法 (#8-提出方法)

3.1.2 ファイル構成

外部から提供していただくファイルは **2種類** です。

ファイル	説明
`<quiz-id>.json`	問題データ（問題・選択肢・正解・解説）
メタデータ情報	クイズの名前・カテゴリ・難易度など

メタデータは提出時に情報として記載いただければ、管理者側で `quizSets.json` に登録します。

> リポジトリ内では、管理者が `src/data/` を編集元として管理し、ビルド時に `public/data/` へコピーします。外部提供者は **JSON ファイル本体とメタデータ情報** を提出すれば十分です。

3.1.2.1 配置イメージ

```
提出物
  <quiz-id>.json                          <-  提供いただくファイル
  メタデータ情報                          <-  Issue / PR 本文などで提出

管理者側の配置
  src/data/<カテゴリ>/<quiz-id>.json      <-  編集元
  src/data/quizSets.json                  <-  管理者が更新
```

カテゴリ例: `github-copilot/`, `clean-architecture/`, `typescript/`, `aws/`

3.1.3 問題ファイル仕様

3.1.3.1 1. ファイル全体の構造

```
(#cb25-1){
(#cb25-2)  "questions": [
(#cb25-3)    { /* 問題オブジェクト 1 */ },
(#cb25-4)    { /* 問題オブジェクト 2 */ }
(#cb25-5)  ]
(#cb25-6)}
```

トップレベルは `questions` キーを持つオブジェクトのみです。
追加のキーは**禁止**（`additionalProperties: false`）。

3.1.3.2 2. 問題オブジェクト仕様

```
(#cb26-1){
(#cb26-2)  "id": 1,
(#cb26-3)  "question": "問題文をここに記述する",
(#cb26-4)  "options": [
(#cb26-5)    { "id": "A", "text": "選択肢A" },
(#cb26-6)    { "id": "B", "text": "選択肢B" },
(#cb26-7)    { "id": "C", "text": "選択肢C" },
(#cb26-8)    { "id": "D", "text": "選択肢D" }
(#cb26-9)  ],
(#cb26-10)  "correctAnswer": "B",
(#cb26-11)  "explanation": "なぜBが正解なのかの詳しい解説をここに記述する。"
(#cb26-12)}
```

3.1.3.2.1 フィールド定義

フィールド	型	必須	制約	説明
`id`	number	[OK]	1 以上の整数、クイズ内連番	問題の番号（1, 2, 3...）
`question`	string	[OK]	空文字禁止	問題文
`options`	array	[OK]	必ず 4 要素（A/B/C/D 全て）	選択肢の配列
`options[].id`	string	[OK]	`"A"`, `"B"`, `"C"`, `"D"` のいずれか	選択肢の識別子
`options[].text`	string	[OK]	空文字禁止	選択肢の本文
`correctAnswer`	string	[OK]	`"A"`, `"B"`, `"C"`, `"D"` のいずれか	正解の選択肢 ID
`explanation`	string	[OK]	空文字禁止	正解の解説

> [Warning] 各選択肢オブジェクトに指定できるプロパティは `id` と `text` のみです。追加プロパティは許可されません。

3.1.3.3 3. 完全な記述例

```
(#cb27-1){
(#cb27-2)  "questions": [
(#cb27-3)    {
(#cb27-4)      "id": 1,
(#cb27-5)      "question": "GitHub Copilot でより正確なコード補完を得るために効果的なコメントの書き方はどれですか？",
(#cb27-6)      "options": [
(#cb27-7)        { "id": "A", "text": "曖昧なコメント（例：「ユーザーを処理する」）" },
(#cb27-8)        { "id": "B", "text": "具体的なコメント（例：「最終ログインが30日以上前のユーザーを非アクティブとしてマークする」）" },
(#cb27-9)        { "id": "C", "text": "コメントなし" },
(#cb27-10)        { "id": "D", "text": "英語のコメントのみ" }
(#cb27-11)      ],
(#cb27-12)      "correctAnswer": "B",
(#cb27-13)      "explanation": "具体的なコメントを書くことで、Copilot はより正確なコードを提案できます。曖昧なコメントでは意図が伝わりにくく、不正確な提案になる可能性があります。問題の範囲・対象・処理内容を明示することが重要です。"
(#cb27-14)    },
(#cb27-15)    {
(#cb27-16)      "id": 2,
(#cb27-17)      "question": "JSON スキーマの `additionalProperties: false` が意味することは何ですか？",
(#cb27-18)      "options": [
(#cb27-19)        { "id": "A", "text": "追加のプロパティを自由に加えられる" },
(#cb27-20)        { "id": "B", "text": "スキーマに定義されていないプロパティは許可されない" },
(#cb27-21)        { "id": "C", "text": "プロパティ数に上限がない" },
(#cb27-22)        { "id": "D", "text": "必須プロパティが存在しなくてよい" }
(#cb27-23)      ],
(#cb27-24)      "correctAnswer": "B",
(#cb27-25)      "explanation": "`additionalProperties: false` は、スキーマに定義されていないプロパティをオブジェクトに含めることを禁止します。これにより、データの形式を厳密に管理できます。"
(#cb27-26)    }
(#cb27-27)  ]
(#cb27-28)}
```

3.1.4 メタデータ仕様

取り込み依頼時に以下の情報を提供してください。内容を元に管理者が `quizSets.json` に登録します。

3.1.4.1 1. フィールド定義

フィールド	型	必須	説明	例
`id`	string	[OK]	一意の ID（kebab-case、英小文字・数字・ハイフンのみ）	`"typescript-basics"`
`name`	string	[OK]	クイズ一覧に表示される名前	`"TypeScript 基礎"`
`description`	string	[OK]	学習内容の概要（1〜500 文字、50〜200 文字推奨）	`"TypeScript の型システムと基本文法を学ぶクイズです"`
`category`	string	[OK]	カテゴリ名	`"TypeScript"`, `"GitHub"`, `"AWS"`
`icon`	string	[OK]	絵文字または短いアイコン文字列	`" "`, `" "`, `" "`
`questionCount`	number	[OK]	問題数（問題ファイルの実際の件数	`20`

フィールド	型	必須	説明	例
			と一致させる）	
`difficulty`	string	[OK]	難易度（下記一覧から選択）	`"intermediate"`
`dataPath`	string	[OK]	問題ファイルの保存場所（カテゴリ/ファイル名.json）	`"typescript/typescript-basics.json"`
`parentId`	string|null	[OK]	シリーズ親の ID（単体なら `null`）	`null`
`group`	string|null	[OK]	シリーズのグループ識別子（単体なら `null`）	`null`
`level`	number	[OK]	`1` = 単体またはシリーズ親、`2` = シリーズの子	`1`
`order`	number	[OK]	グループ内の表示順（1 から	`1`

フィールド	型	必須	説明	例
			始まる正の整数）	
`estimatedLearningTime`	string	—	推定学習時間（任意）	`"10〜15 分"`
`topics`	string[]	—	学習トピック一覧（任意）	`["型注釈", "インターフェース", "ジェネリクス"]`

3.1.4.2 2. difficulty の選択肢

値	対象
`"beginner"`	用語の定義・基本概念の確認
`"intermediate"`	設定・構成・エラー解決の理解
`"advanced"`	システム全体の把握・複合的な分析
`"beginner to intermediate"`	初級〜中級の混在セット
`"beginner to advanced"`	初級〜上級の幅広いセット

3.1.4.3 3. メタデータ記述例（単体セット）

```
(#cb28-1){
(#cb28-2)  "id": "typescript-basics",
(#cb28-3)  "name": "TypeScript 基礎",
(#cb28-4)  "description": "TypeScript の型システム・インターフェース・ジェネリクスを中心
           に学ぶクイズです。JavaScript 経験者を対象としています。",
(#cb28-5)  "category": "TypeScript",
(#cb28-6)  "icon": " ",
(#cb28-7)  "questionCount": 20,
(#cb28-8)  "difficulty": "beginner to intermediate",
(#cb28-9)  "dataPath": "typescript/typescript-basics.json",
(#cb28-10)  "parentId": null,
(#cb28-11)  "group": null,
(#cb28-12)  "level": 1,
(#cb28-13)  "order": 1
(#cb28-14)}
```

3.1.5 シリーズ（階層）構成

複数のステップに分かれるクイズシリーズを作成することができます。

3.1.5.1 1. 構成ルール

- **親エントリ**（`level: 1`）：シリーズ全体のラベルとして機能する。`dataPath` は `""` または `null` にする
- **子エントリ**（`level: 2`）：実際の問題ファイルを持つ。`parentId` に親の `id` を指定する
- `group` フィールドで親・子を同一グループとして識別する（親・子で同じ値を設定）

3.1.5.2 2. シリーズのメタデータ例

```
(#cb29-1)[
(#cb29-2)  {
(#cb29-3)    "id": "aws-solutions-architect",
(#cb29-4)    "name": "AWS ソリューションアーキテクト",
(#cb29-5)    "description": "AWS ソリューションアーキテクト試験対策シリーズ",
(#cb29-6)    "category": "AWS",
(#cb29-7)    "icon": " ",
(#cb29-8)    "questionCount": 0,
(#cb29-9)    "difficulty": "intermediate",
(#cb29-10)    "dataPath": "",
(#cb29-11)    "parentId": null,
(#cb29-12)    "group": "aws-solutions-architect",
(#cb29-13)    "level": 1,
(#cb29-14)    "order": 1
(#cb29-15)  },
(#cb29-16)  {
(#cb29-17)    "id": "aws-solutions-architect-step1",
(#cb29-18)    "name": "Step 1: IAM とセキュリティ",
(#cb29-19)    "description": "IAM ユーザー・ポリシー・ロールを中心にしたセキュリティの基
礎",
(#cb29-20)    "category": "AWS",
(#cb29-21)    "icon": " ",
(#cb29-22)    "questionCount": 15,
(#cb29-23)    "difficulty": "beginner to intermediate",
(#cb29-24)    "dataPath": "aws/aws-solutions-architect-step1.json",
(#cb29-25)    "parentId": "aws-solutions-architect",
(#cb29-26)    "group": "aws-solutions-architect",
(#cb29-27)    "level": 2,
(#cb29-28)    "order": 1
(#cb29-29)  },
(#cb29-30)  {
(#cb29-31)    "id": "aws-solutions-architect-step2",
(#cb29-32)    "name": "Step 2: VPC とネットワーク",
(#cb29-33)    "description": "VPC・サブネット・セキュリティグループ・ルーティングの理解",
(#cb29-34)    "category": "AWS",
(#cb29-35)    "icon": " ",
(#cb29-36)    "questionCount": 20,
(#cb29-37)    "difficulty": "intermediate",
(#cb29-38)    "dataPath": "aws/aws-solutions-architect-step2.json",
(#cb29-39)    "parentId": "aws-solutions-architect",
(#cb29-40)    "group": "aws-solutions-architect",
(#cb29-41)    "level": 2,
```

```
(#cb29-42)    "order": 2
(#cb29-43)  }
(#cb29-44)]
```

3.1.6 命名規則

対象	ルール	良い例	悪い例
クイズ ID	英小文字・数字・ハイフンのみ（kebab-case）	`typescript-basics`	`TypeScriptBasics`, `typescript_basics`
ファイル名	`<quiz-id>.json`	`typescript-basics.json`	`TypeScriptBasics.json`
カテゴリフォルダ	英小文字・ハイフン	`typescript/`, `github-copilot/`	`TypeScript/`, `GitHub Copilot/`
`options[].id`	大文字アルファベット 1 文字	`"A"`, `"B"`, `"C"`, `"D"`	`"a"`, `"1"`, `"Option1"`

3.1.7 難易度ガイドライン

難易度	推奨割合	問題のタイプ例
beginner	約 15%	用語の定義（「〇〇とは何ですか？」）、基本コマンドの目的
intermediate	約 70%	設定の意味・エラーの解決方法・動作の違いを問う問題
advanced	約 15%	システム全体の把握、複数概念の組み合わせ、トレードオフ分析

3.1.7.1 良い問題の条件

- 問題文が一意に解釈できる（曖昧さがない）

- 不正解選択肢が「それらしく見えるが誤り」である（識別力がある）
- 解説が「なぜ正解か」「なぜ他は誤りか」を具体的に説明している
- 1 問で 1 つの概念・事実を問う（複合問題は上級向けのみ）

3.1.8 バリデーション

リポジトリをクローンした環境では、以下のコマンドで仕様への適合を検証できます。

```
(#cb30-1)# メタデータの検証（quizSets.json 全体）
(#cb30-2)npm run validate:metadata
(#cb30-3)
(#cb30-4)# 問題ファイルの検証（単一ファイル指定）
(#cb30-5)npm run validate:quiz -- src/data/<カテゴリ>/<quiz-id>.json
(#cb30-6)
(#cb30-7)# 問題ファイルの検証（設定ファイルの questionGlobs に一致するもの）
(#cb30-8)npm run validate:quiz
```

> 単一ファイルはコマンド引数で直接指定できます。複数ファイルをまとめて検証対象に含めたい場合は、管理者側で `.github/skills-config/quiz-generator/quiz-generator.config.json` の `questionGlobs` を更新してください。

3.1.8.1 JSON スキーマ

スキーマファイルは以下に公開されています（参照のみ）。

スキーマ	パス	用途
問題オブジェクト	`public/schemas/question-schema.json`	各問題の形式検証
問題ファイル全体	`public/schemas/question-set-schema.json`	ファイル全体の形式検証
メタデータ	`public/schemas/quizset-metadata-schema.json`	`quizSets.json` 各エントリの検証

3.1.9 提出方法

3.1.9.1 Option A: GitHub Pull Request（推奨）

1. リポジトリをフォーク: https://github.com/duwenji/spa-quiz-app (https://github.com/duwenji/spa-quiz-app)
2. フォーク先で問題 JSON ファイルを `src/data/<カテゴリ>/` に追加する
3. Pull Request を作成し、本文にメタデータ情報（3-1 節 (#3-1-フィールド定義) の全フィールド）を記載する

3.1.9.2 Option B: Issue での依頼

1. GitHub Issues で新規 Issue を作成する
2. 問題 JSON の内容とメタデータ情報を Issue に記載する
3. 管理者が取り込みを行う

3.1.9.3 提出チェックリスト

- [ ] ファイル名が kebab-case になっている（例: `my-quiz.json`）
- [ ] `id` が 1 から始まる連番になっている
- [ ] `options` が A / B / C / D の 4 つすべて含まれている
- [ ] `correctAnswer` が `options` のいずれかの `id` と一致している
- [ ] `explanation` が空でない
- [ ] `questionCount`（メタデータ）と問題ファイルの実際の問題数が一致している
- [ ] `description` が 50 文字以上 200 文字以内（推奨）
- [ ] ファイルが有効な JSON（構文エラーがない）